

PROJECT-R · DEEPQUANT

Auto-Trade & AI Commentary

A plain-English curriculum on how the system finds trades,
decides on them, executes them, and writes about them.

Generated 15 July 2026 · 10 lessons · every number verified against real recorded data

Auto-Trade & AI Commentary — The Curriculum

A plain-English course on how this app finds trades, decides on them, executes them, and writes about them. No prior trading or coding knowledge needed — every term is explained the first time it appears.

How to read this

Read the lessons in order. Each one builds on the one before it.

Lesson	What you'll learn
01 — The Big Picture	What the whole system is, and one full day in its life
02 — The Words You Need	Options, premiums, lots, OI, R-Factor — the vocabulary, simply
03 — The Data Engine	Where the numbers come from: the 5-minute heartbeat
04 — The Scanner	How the app picks which stocks are worth trading
05 — The AI	The two jobs the AI does: deciding and explaining
06 — How a Trade Happens	A real trade followed step by step, entry to exit
07 — The Safety Rules	The hard limits that no one — not even the AI — can break
08 — Brokers and Modes	Paper vs approval vs live, and how orders reach a broker
09 — Record-Keeping and Review	Every decision is written down; how the app grades itself

Lessons 03–09 each end with an **"Under the hood"** section that maps the lesson's ideas to the actual code — which method in which file does the job — and spells out the maths (formulas checked against real recorded trades). Beginners can skip those sections on a first read; they're there for when you want to see exactly where a rule or a number comes from.

The one-sentence summary

*Every 5 minutes during market hours, the app collects fresh market data, a rule-based scanner shortlists stocks showing unusual activity, an AI looks at the shortlist and manages positions — and **hard-coded safety rules have the final word on every single order.***

The design law

The whole system follows one rule, worth memorizing before lesson 1:

The AI proposes, the code disposes.

The AI can suggest a trade, but a fixed set of safety checks written in code (time window, money caps, loss limits) re-verifies every action. If a check fails, the action is rejected — no matter how confident the AI sounds.

Lesson 01 — The Big Picture

What is this system?

This app trades **stock options** on the Indian market (NSE) with a small, fixed amount of money, in a tightly controlled morning window. It has three main parts working together:

1. **The Data Engine** — a background loop that wakes up every 5 minutes and collects fresh prices and activity data for about 50 large stocks.
2. **The Scanner** — a set of fixed rules (no AI) that reads that data and shortlists stocks where "big money" seems to be quietly building a position.
3. **The AI layer** — an AI that looks at the shortlist, decides whether to actually trade, manages any open trade, and writes a short plain-English note ("commentary") about what it sees.

Around them sits a wall of **safety rules written in code** — entry times, money caps, loss limits — that check every single action. The AI cannot talk its way past them.

One day in the system's life

All times are Indian Standard Time (IST).

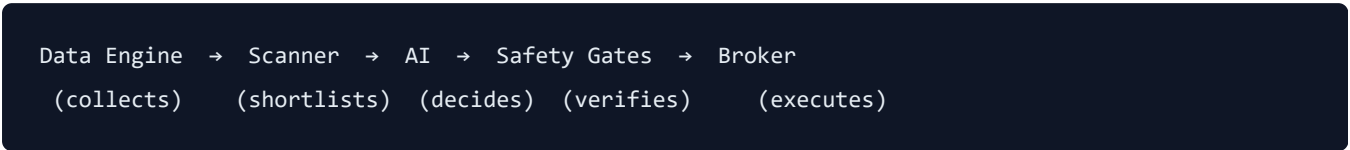
Time	What happens
08:40–09:15	The app quietly logs itself into both brokers (Fyers and Dhan) so tokens are ready before the market opens. No human needed.
09:15	Market opens. The 5-minute data loop starts recording prices and activity.
09:40	The scanner starts running every 5 minutes, looking for unusual activity.
09:45–11:00	The entry window. This is the ONLY time new trades may be opened. The AI reviews the scanner's picks each cycle and may open a trade (max 3 per day).
11:00 onwards	No new trades. The system only <i>manages</i> whatever is open — moving stops, taking profits, cutting losses.
15:12	Forced square-off. Anything still open is closed, no exceptions.
15:30	Market closes. Data recording stops.
After 16:00	The end-of-day scorecard runs automatically: it grades every pick the scanner made today (did it hit its target? its stop?) and stores the report card.

Why only 09:45–11:00 for new trades?

The first 30 minutes of the market are noisy — prices jump around while overnight orders settle. By 09:45 the picture is clearer, but the day's real moves usually start in the morning. Trading only in this window

means: entries are made when signals are freshest, and there's a whole day left for the trade to work — with a hard 15:12 exit so nothing is ever held overnight.

The chain of command



Every arrow is a checkpoint. A stock must pass the scanner's rules to reach the AI. The AI's decision must pass the safety gates to become an order. Only then does anything reach a broker.

What the AI can and cannot do

The AI CAN	The AI CANNOT
Choose to trade one of the scanner's picks, or skip all of them	Invent its own stock, strike, or position size
Exit a trade early if the story changes	Hold past the 15:12 square-off
Move a stop-loss to lock in profit	Move a stop-loss to allow MORE loss (stops may only tighten)
Write the commentary explaining its thinking	Bypass any money or time limit

Next: [Lesson 02 — The Words You Need](#), where every trading term used above (options, stops, lots...) gets a simple definition.

Lesson 02 — The Words You Need

Every term the rest of the course uses, in plain words. Skim it now, come back whenever a word trips you.

Options basics

Option — a contract that lets you profit from a stock's move without buying the stock itself. You pay a small price (the *premium*) for the contract. If the stock moves your way, the premium grows and you sell the contract for more than you paid. If it moves against you, the premium shrinks.

CE (Call option) — the contract you buy when you expect the stock to go **UP**.

PE (Put option) — the contract you buy when you expect the stock to go **DOWN**. Yes — with a PE you *make money when the stock falls*.

Strike — the reference price written into the contract. "HYUNDAI 2040 CE" means a call option on HYUNDAI with strike 2040. The app picks strikes close to the current stock price, because those respond fastest to the stock's moves.

Premium — the price of the option contract itself, per share. This is the number that goes up and down all day, and it's what we actually buy and sell.

Lot / lot size — options are not sold one share at a time; they come in a fixed bundle called a lot. HYUNDAI's lot size is 275, so one lot of a ₹59.50 premium costs $59.50 \times 275 = \text{₹}16,362$. This app always trades exactly **1 lot**.

Expiry — the date the contract stops existing. The app only trades contracts with at least a few days of life left, so they don't decay too fast.

Spot — the actual current price of the stock itself (as opposed to the option's premium). "Spot 2040.9" = the stock trades at ₹2040.90 right now.

Trade management words

Entry — buying the option to open the trade.

Exit — selling it to close the trade. $\text{Profit} = (\text{exit premium} - \text{entry premium}) \times \text{lot size}$.

Stop-loss (SL) — a "get out" line. If the trade goes against us and the price crosses this line, we exit immediately and accept a small loss rather than risk a big one. Every trade here has TWO stop lines: one on the stock's price (spot SL) and one on the option's premium (premium SL). Whichever is hit first wins.

Target — the "take the win" line, same idea in the profit direction.

Square-off — force-closing a position. At 15:12 every open trade is squared off, so nothing is ever held overnight.

Slippage — the small difference between the price you saw when deciding and the price you actually got. The app rejects an entry if the premium moved more than 4% since the scanner quoted it.

Bid-ask spread — the gap between the best buying and selling price. A wide gap means few people are trading that contract, and getting in/out is expensive. The app rejects contracts with a wide spread.

Market-activity words (what the scanner reads)

Volume — how many shares/contracts changed hands. High volume = lots of interest today.

OI (Open Interest) — how many option/futures contracts are currently held open. This is the scanner's favorite clue: when OI *grows*, new money is entering positions. When large OI builds up quietly in one stock, someone big may know (or expect) something.

Futures — a cousin of options: a contract to buy/sell the stock at a set date. Big institutions build positions in futures, so futures OI growth is a strong participation clue.

20-day average — the app compares today's numbers to the stock's own last-20-days norm. "Futures OI 1.35× the 20-day average" means 35% more open contracts than normal for that stock — that's unusual, and unusual is interesting.

R-Factor — this app's home-grown 1-to-8 score of *how unusual* a stock's activity is today (built from OI, volumes, and price-range data). Higher = more unusual. Important: R-Factor measures **participation, not direction or timing** — a high score says "big money is here", not "buy now". The scanner's other checks decide direction and timing.

Breakout — when a stock's price pushes past the range it traded in during the opening minutes. A *confirmed* breakout suggests the move has real force.

VWAP — the day's average trade price weighted by volume. Price above VWAP all day = buyers in control (roughly).

Supertrend — a simple trend indicator drawn from recent candles; used as a second opinion on direction.

Candle (5-minute) — a summary of 5 minutes of trading: the open, highest, lowest, and closing price of that little window. The data engine stores the whole day as 5-minute candles.

App words

Paper trade — a fully simulated trade: real market prices, real rules, no real money. The app is running in paper mode while the strategy is proven.

Scanner / picks — the rule-based screen (Lesson 04) and the shortlist it produces each cycle.

Position guard — the deterministic (non-AI) watchdog that checks every open trade each cycle against its stop/target/square-off lines. Works even if the AI is down.

Kill switch — one setting that instantly stops all new orders.

Next: [Lesson 03 — The Data Engine](#) — where all these numbers actually come from.

Lesson 03 — The Data Engine

Nothing smart can happen without fresh, trustworthy data. This lesson explains the plumbing.

The 5-minute heartbeat

At the center is the **Fyers poller** (`lib/fyers/poller.ts`) — a loop that starts when the server boots and ticks **every 5 minutes, around the clock**, with no browser page open and no human involved.

What a tick does depends on the time of day:

When	What the tick does
08:40–09:15 on a trading day	Token warm-up : logs into Fyers and Dhan so access tokens exist before the open. Retries are free — the loop just tries again 5 minutes later.
09:15–15:30 (market hours)	Records data (below), then runs the autonomous capture : scanner → auto-trade pass → commentary.
After ~16:00	Runs the end-of-day scorecard once (grades the day's picks).
Any other time	Does nothing and goes back to sleep.

What gets recorded every 5 minutes

For roughly 50 large F&O stocks (the ones with active futures and options):

- **5-minute candles** of the stock price (and its futures contract) — open/high/low/close/volume for each 5-minute window. Stored in the `fyers_candles` table, kept for the last **20 trading sessions** so we can replay and study past days.
- **Open Interest (OI) snapshots** — how many contracts are held open, so we can see OI *growing or shrinking during the day*, not just at day-end.
- **Leaderboard ranks** — every cycle the stocks are ranked by activity, and the rank is saved. That lets us spot "climbers" — stocks moving up the activity board hour by hour.

Where the data comes from

- **Fyers** — the primary source for intraday candles and OI recording.
- **Dhan** — a second broker API used for live quotes and option-chain detail (and as an execution venue, Lesson 08).
- **NSE bhavcopy** — the exchange's official end-of-day file, downloaded once per day after it's published (post-midnight). This gives every stock's official daily OI, volume, and turnover — the baseline the "20-day average" comparisons are built on.

Rate limits — why the app is deliberately slow and polite

Broker APIs allow only a few requests per second. The app **never fires requests in parallel** at a broker; it spaces them out (about 3 per second to Fyers, 1 per second to Dhan's quote APIs). Trying to be faster gets you blocked, and being blocked during market hours means blind trading — so politeness is a safety feature.

Tokens — the daily keys

Broker APIs need an access token that expires every day. The app generates these itself using a stored PIN + a TOTP secret (the same 6-digit-code trick as an authenticator app), saves the token to disk, and renews it before it expires. This is why the system can run for weeks with nobody logging in.

Sharing one data stream between many viewers

If five people open the live page, the app does NOT make five sets of broker calls. Quote responses are cached for ~6.5 seconds and shared — so N viewers cost the same broker traffic as one. The scanner always bypasses this cache (`fresh: true`) because real trade decisions must never read a stale price.

One honest rule about data

The app never invents a number. If a quote is missing, a paper trade simply fails rather than being filled at a made-up price; if a candle didn't arrive, the cycle logs "no usable candles" and keeps the previous ones. Every number you see on a page traces back to a real broker or exchange response.

Under the hood — the code behind this lesson

Skippable if you don't read code — but here is exactly where each idea lives.

Concept	Method (file)	What it does
The heartbeat starts	<code>startFyersPoller()</code> in <code>lib/fyers/poller.ts</code>	Called once at server boot (<code>instrumentation.ts</code>); kicks off the loop
Ticks land on 5-min marks	<code>scheduleNextTick()</code> (same file)	Computes how long to sleep so the next tick aligns with the next 5-minute boundary
One full tick	<code>runFyersCycle()</code>	Downloads candles + OI for the universe, stores them, then runs scanner → auto-trade → commentary
"Is the data complete?"	<code>hasRequiredEqBar()</code>	Before acting, checks the expected latest 5-min bar actually arrived — no acting on holes
Morning token minting	<code>inWarmupClockWindow()</code> + <code>runTokenWarmup()</code>	The 08:40–09:15 warm-up; every tick in the window is a free retry

Concept	Method (file)	What it does
Daily keys	<code>getFyersAccessToken()</code> / <code>getDhanAccessToken()</code> (<code>lib/fyers/auth.ts</code> , <code>lib/dhan/auth.ts</code>)	Idempotent: return the cached token instantly while valid; only one caller mints at a time (promise lock); token saved to disk
Candle storage	<code>lib/fyers/candle-store.ts</code>	Upserts each 5-min bar; keeps the last 20 sessions
Shared viewer cache	<code>app/api/live/_lib/quote-response-cache.ts</code>	The 6.5-second cache + "coalescing" (simultaneous identical requests share one broker call)
Evening report card	<code>runEodScorecard()</code> in <code>poller.ts</code>	After 16:00, calls <code>reviewToday()</code> once per day (maths in Lesson 09)

The one piece of maths here — the candle bucket. Every timestamp is rounded DOWN to its 5-minute slot: `bucketTs = floor(time ÷ 300) × 300` (300 seconds = 5 minutes). All ticks inside a slot form one candle: its **open** is the first price, **high** the highest, **low** the lowest, **close** the last. That's the entire trick behind every candle chart you've ever seen.

Next: [Lesson 04 — The Scanner](#) — how 50 stocks become 2–3 picks.

Lesson 04 — The Scanner

The scanner (`lib/trade-suggest/engine.ts`) is the part that answers: "out of ~50 stocks, which 2–3 are worth the AI's attention right now?" It is pure rules — no AI, same input always gives the same output. It runs every 5 minutes from 09:40 to 11:00.

The idea it hunts for

Before a stock makes a big move, someone usually *positions* for it: open interest builds up beyond normal, options activity swells, the price starts pressing against its morning range. The scanner looks for that **fingerprint of quiet accumulation** — and then insists the price itself confirms the direction before suggesting a trade.

The gates, in order

Think of it as a funnel. A stock must pass EVERY gate:

1. **Unusual participation** — R-Factor must be at least **3.6** (on the 1–8 scale), with directional agreement between its factors. Translation: the stock's activity today must be clearly abnormal, and the clues must point the same way.
2. **Real OI build** — futures OI at least **1.1× the 20-day average**, and NSE's live feed showing combined OI up at least **5%** today. New money must actually be entering.
3. **Options-led path** (for stocks where the buildup is in options rather than futures): options must be at least **10%** of the activity and the options premium traded must be at least **₹5 crore** — big enough to matter, not retail noise.
4. **Tradeable contract** — the option the app would actually buy must be liquid: bid-ask spread within limits, enough days to expiry, and **one lot must fit the capital budget**.
5. **Direction & timing checks** — where is price versus the opening range, VWAP, and Supertrend? A stock "extended" too far from its open gets penalized (chasing a move that already happened is how you buy the top).
6. **A survivable plan** — the scanner computes the full trade plan (entry, spot stop, spot target, premium stop, premium target) and requires roughly **2× reward-to-risk**, with the worst case capped near **₹1,500 per lot** and the goal near **₹5,000 per lot**. If the math can't offer that, no pick.

Survivors are scored and ranked; at most **7** appear on the page, and in practice 1–3 per cycle.

What a pick looks like

A pick is a complete, ready-to-execute plan — not a vague tip:

PATANJALI 355 PE (bearish) — spot 353.3, stop if spot rises to 353.3, target spot 341.9; premium ₹21.55, premium stop ₹19.95, premium target ₹26. Reasons: R-Factor 7.19, futures OI 1.35× 20-day avg, breakout confirmed...

Every pick is saved to the `trade_suggestions` table with all its reasons — so weeks later we can check exactly what the scanner saw and how it graded out.

What the scanner does NOT do

- It does **not** place trades. It only produces candidates.
- It does **not** predict. It reads today's positioning and today's price confirmation; it can be wrong (see the TATAELXSI story in Lesson 09).
- It does **not** re-suggest smartly: if a pick stays valid, it will keep appearing cycle after cycle. Appearing 22 times does not make it 22× better — it means the conditions kept holding.

Switches waiting for proof

A few extra filters exist in the code but are **switched OFF** until replays over many recorded days prove they help (this discipline matters — a filter that "fixes" one bad day often quietly deletes the winners of other days):

- **TF breakout gate** — only allow picks with a confirmed breakout in the trade's direction.
- **Momentum breakout path** — a 4th entry style for explosive short-covering moves that the accumulation gates skip by design.

The pending decisions live on the [/reminders](#) page.

Under the hood — the code and the maths

Skippable if you don't read code — but this is where every idea above lives, and the maths is worth seeing once because the recorded picks check out against these formulas exactly.

The methods

Concept	Method (file)	What it does
The whole funnel	<code>runTradeSuggest()</code> in <code>lib/trade-suggest/engine.ts</code>	Runs one scan: candidates → gates → scoring → plans → store
The 20-day "normal"	<code>loadFactorBaselines()</code> (same file)	Reads each stock's last-20-days averages (OI, turnover, range) from the bhavcopy tables
R-Factor	<code>computeRFactor()</code> in <code>lib/r-factor/engine.ts</code>	Runs 12 factor signals and blends them (maths below)
OI urgency	<code>computeOiUrgency()</code> in <code>lib/signals/oi-intraday.ts</code>	Scores 0–10 how fast OI is climbing today from the intraday snapshots
Ranking the survivors	<code>computeCompositeScore()</code> in <code>lib/trade-suggest/scoring.ts</code>	The weighted score that orders the picks (maths below)

Concept	Method (file)	What it does
The stock-price plan	<code>buildSpotPlan()</code> (same file)	Derives spot stop and target from the candles (maths below)
Grading past picks	<code>reviewToday()</code> in <code>lib/trade-suggest/review.ts</code>	The scorecard (Lesson 09)

`scoring.ts` is deliberately pure maths — no clocks, no database — so the live scanner and the offline replay harness run the *same* code, and a replay can never accidentally peek into the future.

The maths — R-Factor

Twelve factor signals (OI level vs 20-day, smart-money accumulation, futures OI change, range spread, bid-ask tightness, turnover, breakout, PCR...) each report a strength between 0 and 1 — or "not available" if their data is missing. The blend:

```
rawScore = (w1×f1 + w2×f2 + ...) ÷ (sum of weights of AVAILABLE factors)
R-Factor = 1 + 7 × rawScore           → a number from 1 to 8
```

Dividing by only the *available* weights means missing data (say, no option chain) neither inflates nor deflates the score. Direction (bullish/bearish) is a separate **majority vote** of the directional factors — R-Factor's size and its direction are computed independently.

The maths — the composite pick score

Each component is first squashed to a 0–1 scale, then weighted (weights sum to 1.0):

Component	Weight	In plain words
R-Factor	0.22	How unusual is today's participation
Opening-range breakout	0.20	Has price actually broken out
OI urgency	0.18	How FAST is OI building right now
OI level	0.12	How far above the 20-day norm
Confidence	0.08	Do the factors agree on direction
Sector breadth	0.08	Are same-sector names moving the same way
Order-book imbalance	0.07	Are buyers or sellers heavier, and does it match our side
Setup strength	0.05	The setup classifier's grade

An "extended" stock (already moved far from its open) has its final score multiplied by **0.6** — chasing is penalized, not banned.

The maths — the spot plan (stop and target)

`buildSpotPlan()` for a bullish (CE) pick:

1. **Stop** = the low of the last *completed* 5-min candle (a natural "if it breaks this, the move failed" line). If that's not usable, fall back to the opening-range boundary. (For PE picks: mirror image — the last candle's high.)
2. **Noise floor**: if that stop is closer than **0.35%** of the entry price, push it out to 0.35% — a stop inside normal 5-minute wobble is a guaranteed pointless stop-out.
3. **Target** = entry + 2 × the risk (`TARGET_RR = 2`): you only take trades where the win is twice the planned loss.

```
risk    = entry - stop          (at least 0.35% of entry)
target = entry + 2 × risk      (CE;  entry - 2 × risk for PE)
```

The maths — the premium plan (the rupee lines)

For the option itself (per-share premium, lot size L):

```
premium stop  = the TIGHTER of:  LTP × (1 - 40%)   and   LTP - 1500 ÷ L
premium target = LTP + 5000 ÷ L
```

That first line is why no lot can lose much more than **₹1,500**, and the second aims each lot at **+₹5,000**. Check it against a real recorded pick — MANKIND on 15 Jul (premium ₹71.60, lot size 250):

```
stop    = 71.60 - 1500 ÷ 250 = 71.60 - 6.00 = ₹65.60   ✓ (recorded: 65.60)
target  = 71.60 + 5000 ÷ 250 = 71.60 + 20.00 = ₹91.60  ✓ (recorded: 91.60)
```

Next: [Lesson 05 — The AI](#) — what happens to these picks.

Lesson 05 — The AI

The AI has exactly two jobs, and it does them once per 5-minute cycle:

1. **Decide** — look at the scanner's picks and any open position, and choose: enter, hold, tighten a stop, exit, or do nothing.
2. **Explain** — write the **trade commentary**: a short, plain-English note on what the market is doing and why it acted (or didn't).

One brain per cycle — never two

Each cycle gets exactly **one** AI analysis. When the auto-trade decision pass runs, the note it writes IS the commentary for that cycle. The standalone commentary writer only steps in as a fallback (auto-trade off, kill switch on, nothing to decide, or the AI call failed). This is a deliberate rule: two AIs analyzing the same scan can disagree, and a disagreeing pair is worse than either alone.

How the decision pass works

The AI is given a structured briefing: today's settings and limits, the open position (if any), the scanner's current picks with all their reasons and plans, and fresh quotes. It then works with **tools** — small functions it may call, such as "get a fresh quote", "place the entry order", "move the stop", "exit now".

Here is the critical part: **every tool that touches money re-runs the safety gates in code** (Lesson 07). The AI saying "place the order" starts a verification, not an order. If any gate fails, the tool refuses and tells the AI why. A confused or overconfident AI cannot spend a rupee the rules don't allow.

The AI also cannot choose its own instrument: **only scanner picks are tradeable**, always 1 lot, with the scanner's plan attached. Its real decision surface is narrow on purpose: *which pick (if any), and when to get out*.

What the commentary looks like

The commentary style is deliberately decisive and jargon-free — one or two confident calls with explicit exits, not a wall of numbers. A real example of the format (from 15 Jul):

11:00 — market leaning up, 30 of 49 names higher, but the one clean bearish setup still passes every gate at the buzzer. PATANJALI — TRADE NOW ...entry, stop, target, and the reason in plain words.

Notes are stored in the `trade_commentary` table and shown on the `/trade-commentary` page; they're also broadcast to Telegram (Lesson 09).

Which AI models are used

Two providers are wired in, selectable in settings:

- **Azure OpenAI** — proven tool-calling behavior.
- **MiMo** — a reasoning model (thinks step-by-step before answering); needs a generous token budget.

The exact instructions (the "system prompt") given to the AI are **versioned**: every time the prompt text changes, a new version is recorded in the `prompt_versions` table, and every commentary row records which version wrote it. You can read the full prompts on the [/prompts](#) page. The code is always the source of truth for prompts — the database is history, never an override.

What if the AI is down?

Trading safety never depends on the AI being up:

- The **position guard** (deterministic code) runs BEFORE the AI every cycle — stops, targets, and the 15:12 square-off fire with or without an AI.
- No AI = no new entries that cycle (entries need a proposal). Fail-closed: when in doubt, do nothing.

The AI's limits, summarized

Question	Answer
Can it pick a stock the scanner didn't?	No
Can it trade 2 lots because it's confident?	No — always 1 lot
Can it widen a stop to "give the trade room"?	No — stops only tighten
Can it enter at 11:05 because the setup is great?	No — window ends 11:00
Can it skip all picks and do nothing?	Yes — and often should

Under the hood — the code behind this lesson

Concept	Method (file)	What it does
One decision cycle	<code>runAutoTradePass()</code> in <code>lib/auto-trade/engine.ts</code>	Orchestrates the pass: position guard first, then the AI conversation; also prevents two passes overlapping
The AI's briefing	<code>buildInitialDecisionContext()</code> in <code>lib/auto-trade/tools/execute.ts</code>	Assembles settings, limits, open positions, scanner picks, and fresh quotes into the structured context the AI reads (helpers: <code>buildAccountState()</code> , <code>buildScanContext()</code> , <code>buildOpenPositionsContext()</code>)
Running a tool call	<code>executeAutoTradeTool()</code> (same file)	The dispatcher: when the AI calls a tool, this runs it — and every money-touching tool re-runs the gates in code before acting

Concept	Method (file)	What it does
The tool list	<code>lib/auto-trade/tools/defs.ts</code>	The fixed menu of what the AI may call — nothing else exists for it
Talking to the AI	<code>lib/auto-trade/decision/providers.ts</code>	Two loops: Azure OpenAI (Responses API) and MiMo (chat-completions). Same tools, same rules
The AI's instructions	<code>lib/auto-trade/decision/system-prompt.ts</code>	The auto-trader prompt; it reuses the commentary's writing rules verbatim so there's ONE source of truth for style
Fallback commentary	<code>runAndStoreCommentary()</code> → <code>generateCommentary()</code> in <code>lib/ai-commentary/run.ts</code> / <code>generate.ts</code>	The standalone narrator (prompt: <code>COMMENTARY_SYSTEM</code>) used only when the auto-trade pass didn't produce a note
Prompt history	<code>lib/prompts/store.ts</code>	Auto-records a new version whenever the prompt text changes; read-only history behind the /prompts page

There's no hidden maths in this layer — deliberately. The AI's inputs are the scanner's numbers (Lesson 04) and its actions are bounded by the gates' arithmetic (Lesson 07). The AI contributes judgment, not calculations.

One implementation detail worth knowing: MiMo is a *reasoning* model — it "thinks" in tokens before answering, so the code gives it a generous token budget and reads its final answer field. Every pass records its token usage in `auto_decisions`, so the cost of every decision is visible.

Next: [Lesson 06 — How a Trade Happens](#) — a real trade, minute by minute.

Lesson 06 — How a Trade Happens

The best way to understand the machine is to watch it work. Everything below is a **real recorded day — 15 July 2026** (paper mode: simulated fills at real market prices). Settings that day: max 3 trades, ₹25,000 capital cap, ₹2,500 daily-loss halt, entries 09:45–11:00.

The lifecycle, in general

```
scanner pick → AI proposes entry → GATES verify → order placed → fill recorded
→ every 5 min: position guard checks stop/target/square-off
→ exit (stop, target, AI decision, or 15:12) → P&L booked → scorecard
```

09:48 — Trade 1: HYUNDAI 2040 CE (a controlled loss)

The scanner had surfaced HYUNDAI (bullish): R-Factor 4.13, options OI up 41%, breakout confirmed. The AI proposed entry; the gates verified: window open ✓, 0 trades so far ✓, cost ₹16,362 under the ₹25,000 cap ✓, premium moved only 2.67% since the scan quote (under the 4% slippage cap) ✓. **Filled at ₹59.50.**

The plan: spot stop 2020, spot target 2082.7, premium stop ₹54.05.

It went wrong quickly — the stock drifted down. At **10:03** the position guard saw the premium at ₹52.55, below the ₹54.05 premium stop, and exited immediately. **Loss: –₹1,911.**

Why this was good behavior: the stock kept falling all day (closed at 2004). The slower spot stop wouldn't have triggered until 12:40, at a bigger loss. The premium stop is the tighter rupee-based guard — it did its job.

10:09 — Trade 2: MANKIND 2600 CE (a win, with stop management)

Scanner pick (bullish), cost ₹18,050 — fits the cap now that HYUNDAI is closed. **Filled at ₹72.20.** Plan: spot stop 2585.6, spot target 2614.1.

At 10:40, with the stock up near 2611, the AI called "MOVE SL to 2608" — locking most of the gain. The gate for stop moves checked: 2608 is HIGHER than the old stop (tighter for a bullish trade) ✓ — allowed. (A move DOWN would have been refused.)

At **10:50** the guard confirmed spot 2625.5, past the 2614.1 target → exited at ₹88.15. **Profit: +₹3,988.**

10:21 — the trade that DIDN'T happen (gates earning their keep)

While MANKIND was open, the scanner surfaced PATANJALI (bearish, ₹19,081 per lot). The AI wanted it. The capital gate said: ₹18,050 already deployed + ₹19,081 = ₹37,131 > ₹25,000 → **blocked**. The AI noted "capital locked" in its commentary and waited.

11:00 — Trade 3: PATANJALI 355 PE (patience pays)

MANKIND closed at 10:50, freeing the capital. At 11:00 — the last minute of the window — PATANJALI still passed every gate, now as a *fresher* plan (the stock had fallen further, so a new strike and deeper target).

Filled at ₹21.35, cost ₹22,951.

The stock kept sliding. At 11:23 the AI locked the stop at breakeven (again: tightening — allowed). At **11:28** the premium hit its ₹26 target → exited at ₹27.40. **Profit: +₹6,504.**

A replay later proved the block at 10:21 was lucky-good: the early plan's target would have paid about +₹5,000, and the no-re-entry rule would then have forbidden the 11:00 trade that made +₹6,504.

The day's ledger

Time	Trade	Result	Running total
10:03	HYUNDAI CE — premium stop	−₹1,911	−₹1,911
10:50	MANKIND CE — spot target	+₹3,988	+₹2,077
11:28	PATANJALI PE — premium target	+₹6,504	+₹8,581

Three trades (the daily max), never more than ₹22,951 deployed at once, the running loss never near the −₹2,500 halt, everything closed hours before square-off. After 11:00 the AI's commentary switched to "entries maxed at 3/3 — managing only", and after 11:28 there was nothing left to manage.

What to notice

- **Every entry passed through the gates**, and one attempted entry was blocked — the block turned out to be the most profitable "decision" of the day.
- **Exits were mechanical.** All three exits were fired by the position guard hitting a pre-planned line, not by anyone's mood.
- **Losses are small by design.** The one loss (−₹1,911) was capped by a stop decided *before* entry, when nobody was emotional.

Under the hood — the code and the maths

The methods

Step in the story	Method (file)	What it does
Placing the entry	<code>placeEntryOrder()</code> in <code>lib/auto-trade/execution.ts</code>	Re-runs the gates, writes the trade + order rows to the database FIRST, then sends via the broker adapter

Step in the story	Method (file)	What it does
Re-anchoring the rupee lines	<code>backstopsFromFill()</code> (same file)	Once the real fill price is known, recomputes the premium stop/target from it (maths below)
Getting out	<code>exitTrade()</code> (same file)	Sends the sell order to the venue the trade opened on, records the fill and the reason
The watchdog	<code>runPositionGuard()</code> in <code>lib/auto-trade/risk/position-guard.ts</code>	Every cycle: checks each open trade against premium stop/target, spot stop/target, and the 15:12 square-off — exits the moment one is crossed
Crash recovery	<code>reconcileUnresolvedOrders()</code> in <code>execution.ts</code>	On restart, any order the app sent but never heard back about is checked against the broker

The maths — re-anchored stops (watch it match the real day)

The scanner's premium lines were computed from the *scan* quote. The actual fill can differ slightly, so the lines are recomputed from the fill:

```
premium stop = the TIGHTER of: fill × 0.60 and fill - 1500 ÷ lotSize
premium target = fill + 5000 ÷ lotSize
```

HYUNDAI on 15 Jul: fill ₹59.50, lot size 275.

```
stop = 59.50 - 1500 ÷ 275 = 59.50 - 5.45 = ₹54.05
```

₹54.05 is *exactly* the stop that fired at 10:03 ("premium stop hit ₹52.55 ≤ ₹54.05"). The formula on this page is the formula that took the loss.

The maths — profit and loss

```
P&L = (exit premium - entry premium) × lot size × lots
```

All three real trades, checked:

Trade	Calculation	Result
HYUNDAI CE	$(52.55 - 59.50) \times 275 \times 1$	-₹1,911 ✓
MANKIND CE	$(88.15 - 72.20) \times 250 \times 1$	+₹3,988 ✓
PATANJALI PE	$(27.40 - 21.35) \times 1075 \times 1$	+₹6,504 ✓

(₹6,503.75 rounds to ₹6,504 — the app rounds booked P&L to the rupee.)

Why "database first, broker second" matters

`placeEntryOrder()` writes its record *before* sending the order. If the app crashed mid-send, restart finds a row saying "I may have sent this" and `reconcileUnresolvedOrders()` asks the broker for the truth. The reverse order (send first, record after) could leave a real position the app doesn't know it owns — the one kind of surprise this design refuses to allow.

Next: [Lesson 07 — The Safety Rules](#) — the complete list of gates that made this day behave.

Lesson 07 — The Safety Rules

This is the heart of the system's trustworthiness. Every rule below lives in code (`lib/auto-trade/risk/gates.ts` and the position guard), runs on every order attempt, and cannot be talked around — not by the AI, not by a bad prompt, not by a bug in a page.

The entry gates

Before ANY entry order is placed, all of these must pass:

Gate	The rule	Why it exists
Mode	Auto-trade must not be OFF, kill switch must not be on	The operator's master switches
Time window	New entries only 09:45–11:00 IST	Fresh signals + a full day for the trade to work
Square-off horizon	Nothing new at/after 15:12	Nothing held overnight, ever
Trade count	Max 3 trades per day (setting)	A bad day stays a small bad day
Open lots	Max 2 lots open at once	No pyramid of positions
Capital cap	Deployed premium may not exceed ₹25,000 (setting)	Hard budget — the app can't "go big"
Daily loss halt	Realized loss $\geq$ ₹2,500 → no more entries today	Stops revenge-trading after losses
No re-entry	A symbol traded today can't be traded again today	No doubling down on the same story
Slippage	Premium moved > 4% since the scan quote → reject	Don't chase a price that already ran
Liquidity	Bid-ask spread > 8% → reject	Illiquid contracts are traps
Has a stop	The plan must include a spot stop-loss	Unmanaged trades are not allowed
Broker funds	For REAL orders: balance must be verifiable	Never assume money is there

Fail-closed: the most important idea on this page

When the system can't verify something, it refuses — it never assumes.

- A number arrives corrupted (not-a-number)? The gate rejects the whole attempt. (In JavaScript, comparing anything to NaN quietly answers "false" — which would have silently *passed* several checks. The gates catch this explicitly and fail closed.)
- The broker can't report available funds? Real orders are blocked.

- Slippage can't be computed because the scan quote is missing? Blocked.
- No live quote for a paper fill? The fill fails — never a made-up price.

The general principle: **a missing safety answer is a "no", not a "probably fine"**.

Rules for trades already open

- **Stops may only tighten.** A bullish trade's stop may only move UP, a bearish one's only DOWN. "Give it more room" is structurally impossible.
- **The position guard runs before the AI every cycle**, and also under the kill switch. It checks: premium stop, premium target, spot stop, spot target, and the 15:12 square-off. It's plain code — it works when the AI is down, confused, or disabled.
- **Exits are always allowed.** Every time gate applies to entries only; getting OUT of risk is never blocked.

Layers above the gates

- **Two-key rule for live money:** switching to `live` mode requires BOTH the setting on the page AND an environment variable (`AUTO_TRADE_LIVE_ENABLED`) set on the server. One click can never take the system live. (`approval` mode's second key is the human tap on each order.)
- **Kill switch:** one toggle stops all new orders instantly; the guard keeps managing exits.
- **Idempotency keys:** every order carries a unique fingerprint, so a retry after a network hiccup can never place the same order twice.
- **Order APIs vs prompts:** none of these rules exist "in the AI's instructions". The instructions repeat them as guidance, but the enforcement is code — the tool that places orders re-runs the gates itself.

How we know the gates work

A bench script (`scripts/verify-auto-trade.ts`) runs **34 automated checks** over the gates, sizing math, settings clamps, and a quiet engine pass — it's run before trusting any config change. And the replay in Lesson 06 re-ran a real day's entries through the real gate code, byte for byte.

Under the hood — the code behind this lesson

Concept	Method (file)	What it does
The entry gates	<code>checkEntryGates(input)</code> in <code>lib/auto-trade/risk/gates.ts</code>	ONE pure function holding every gate in the table above. Takes all facts as inputs (time, counts, money, quotes), returns allow/deny + the reasons. Pure = no database, no clock — fully testable and replayable
Stops only tighten	<code>checkStopMove(direction, currentStop, newStop)</code> (same file)	Bullish: new stop must be HIGHER than the old; bearish: LOWER. Anything else refused

Concept	Method (file)	What it does
The watchdog	<code>runPositionGuard(date)</code> in <code>lib/auto-trade/risk/position-guard.ts</code>	The deterministic exit loop — premium stop/target, spot stop/target, 15:12 square-off
The proof	<code>scripts/verify-auto-trade.ts</code>	The 34-check bench that exercises the gates with normal, edge, and corrupted inputs

The fail-closed maths (the NaN trap)

This is the sneakiest bit of arithmetic in the whole app, so here it is slowly. In JavaScript, a broken number is `NaN` ("not a number"), and **every comparison with NaN answers false**:

```
NaN > 660 → false      NaN <= -2500 → false
```

Now look at a gate like "block if minuteIST > 660 (past 11:00)". If `minuteIST` arrived corrupted as `NaN`, the comparison answers false — meaning **"not past 11:00" — the gate silently passes!** The same trap would let a corrupted P&L slip past the loss halt.

So before any gate runs, `checkEntryGates` validates every numeric input with `Number.isFinite()`. One corrupt number → the whole attempt is rejected with `corrupt numeric input(s): ... — failing closed`. A broken sensor stops the machine; it never gets to vote "everything's fine".

Next: [Lesson 08 — Brokers and Modes](#) — paper, approval, live, and how an order physically reaches a broker.

Lesson 08 — Brokers and Modes

The four modes

Auto-trade has a mode setting on the `/auto-trade` page. It starts at `off` and must be deliberately changed:

Mode	What happens	Real money?
off	Nothing trades. The scanner and commentary still run.	No
paper	Full simulation: the AI decides, gates verify, "fills" happen at the real live market premium at that moment.	No
approval	The AI proposes REAL orders, but each one waits on the <code>/auto-trade</code> page for a human to tap Approve . The approval re-runs all gates against a fresh quote before sending. Proposals expire after 15 minutes.	Yes — with a human key on every order
live	Fully autonomous real orders. Requires the two-key rule (mode setting + server environment variable).	Yes

Paper mode is not a toy — it exercises the entire pipeline (data → scanner → AI → gates → fills → guard → records) with only the final broker call simulated. Its fills use real quotes; if no quote exists, the fill fails rather than inventing a price. That's why weeks of paper results are meaningful evidence.

The brokers

Two brokers are wired in, through a common adapter interface (`lib/auto-trade/brokers/`):

- **Fyers** — orders via the official `fyers-api-v3` SDK.
- **Dhan** — orders via direct REST calls to their v2 API.
- **Paper** — the simulator adapter, same interface, fills from live quotes.

One broker is active at a time (a setting). Two rules keep multi-broker life sane:

1. **Exits always route to the venue the trade opened on.** A trade opened on Fyers is closed on Fyers, even if the active-broker setting changed midday.
2. **Every order carries an idempotency key** — a unique fingerprint generated when the order is created. If the app retries after a timeout, the broker adapter recognizes the fingerprint and will not double-place.

The path of a real order

```
AI calls place_entry_order
  → gates re-verified in code (Lesson 07)
  → order row written to the database FIRST (with its idempotency key)
  → broker adapter sends it
  → broker's answer (accepted / rejected / fill price) recorded on that row
  → position guard takes over monitoring
```

Writing the record *before* sending means a crash can never leave an order the app doesn't know about — on restart, unreconciled orders are checked against the broker.

An honest warning label

The real Fyers/Dhan **order** APIs are fully wired but have not yet been exercised against a live account (everything so far has been paper). The standing instruction — it's also on the /reminders page — is: **the first approval/live order must be watched manually end-to-end**, with the broker terminal open, confirming the app's recorded fill matches the broker's.

Token & login plumbing (why nobody logs in every morning)

Both brokers need fresh daily access tokens. The poller mints them automatically between 08:40 and 09:15 IST using stored credentials + TOTP (Lesson 03) and stores them on disk. The **/fyers** and **/dhan** pages show token status chips, and a health widget surfaces the result of the morning warm-up. If both were down, trading simply fails closed — no tokens, no orders.

Under the hood — the code behind this lesson

Concept	Method / file	What it does
One shape for all brokers	<code>lib/auto-trade/brokers/adapter.ts</code>	The common interface every venue implements: place order, check status, get funds
The venues	<code>fyers-adapter.ts</code> , <code>dhan-adapter.ts</code> , <code>paper-adapter.ts</code> (same folder)	Fyers via the official SDK, Dhan via direct REST, paper via live quotes — swappable because they share the interface
The order fingerprint	<code>correlationIdForOrder(idemKey)</code> in <code>lib/auto-trade/execution.ts</code>	Hashes the order's unique idempotency key (SHA-256, first 19 chars, prefixed R) into the tag sent to the broker — the same order always produces the same tag, so a retry is recognizable, never a duplicate
Crash recovery	<code>reconcileUnresolvedOrders()</code> (same file)	On restart, asks the broker about any order with no recorded final answer

Concept	Method / file	What it does
The mode & settings	<code>lib/auto-trade/settings.ts</code>	Every setting is parsed with a clamp (e.g., square-off can only be set between 14:00 and 15:20) — even the settings screen can't configure something unsafe

The only maths in this layer is the fingerprint: `hash(idemKey)` is deterministic, so "same intent → same tag" — that single property is what makes network retries safe.

Next: [Lesson 09 — Record-Keeping and Review](#) — how everything is written down and graded.

Lesson 09 — Record-Keeping and Review

A system you can't audit is a system you can't trust. This lesson covers what gets written down (everything) and how the app grades itself.

Every layer leaves a paper trail

Table	What it records
<code>fyers_candles</code>	Every 5-minute candle for ~50 stocks, last 20 sessions — the raw material for replays
<code>oi_intraday</code> / <code>rank_snapshots</code>	OI readings and activity-leaderboard ranks, every 5 minutes
<code>trade_suggestions</code>	Every scanner pick ever made: the full plan, ALL its reasons, and how many cycles it stayed valid
<code>auto_decisions</code>	Every AI pass: what it said, which model, how many tokens — including the passes where it did nothing
<code>auto_trades</code> / <code>auto_orders</code>	Every trade and every order: plans, fills, exit reasons, P&L, timestamps
<code>trade_commentary</code>	Every commentary note, with the exact prompt version that wrote it
<code>prompt_versions</code>	Every historical version of the AI instructions

The practical payoff: any question like "*why did we take that trade?*" is answerable **months later**, from data — not from memory. (That's exactly how the HYUNDAI post-mortem and the 15-Jul replays in this course were done.)

The nightly report card

After 16:00 the poller runs the **end-of-day scorecard** automatically. For every pick the scanner made that day, it walks the recorded candles and grades: did the stock reach the plan's target? Hit its stop? How far did it move in our favor at best, and against us at worst? Where did it close?

Grades accumulate in the same `trade_suggestions` rows. Over weeks this builds the evidence base for every "should we turn on this filter?" decision — rules get promoted on multi-day proof, never on one day's story.

The review pages

Page	What it shows
<code>/auto-trade</code>	Today's live console: mode, open position, pending approvals, kill switch

Page	What it shows
/auto-trade/history	EOD auto-trade results, day by day: each trade, fills, exit reasons, day P&L
/trade-suggest	The scanner's current picks with reasons
/trade-suggest/history	EOD scorecard per day: how every pick graded
/trade-commentary	The AI's notes through the day
/prompts	Full history of the AI instructions
/reminders	Parked decisions: switches waiting for more evidence, one-time checks

Telegram

Commentary and auto-trade alerts are pushed to Telegram, so the day can be followed from a phone. The operator's chat gets them by default; additional viewers are added by an explicit allowlist of chat IDs (each viewer must start the bot once). Viewers receive broadcasts only — no controls.

Two lessons the records already taught us

These are worth internalizing, because they shape how the system is tuned:

1. **R-Factor measures participation, not direction or timing.** TATAELXSI (15 Jul) held R-Factor 4.2 with heavy OI all day — and went nowhere, sideways to slightly up against a bearish pick. High R alone is a crowd gauge, not an entry signal; that's why entry rules are separate gates.
2. **Fixing yesterday's loss can delete tomorrow's win.** A filter that would have skipped the HYUNDAI loss (extended-from-open) was tested by replay — it also deleted PATANJALI's +₹6,504. Every proposed rule gets replayed across ALL recorded days before it's trusted. One-day evidence is an anecdote, not a rule.

Under the hood — the code and the maths

Concept	Method (file)	What it does
The nightly grader	<code>reviewToday()</code> in <code>lib/trade-suggest/review.ts</code>	Grades every pick from its own suggestion moment onward (maths below)
Its trigger	<code>runEodScorecard()</code> in <code>lib/fyers/poller.ts</code>	First off-hours tick after 16:00 IST, once per day
Trade history reads	<code>getAutoTradeDates()</code> / <code>getTradesByDate()</code> in <code>lib/auto-trade/store.ts</code>	Feed the <code>/auto-trade/history</code> page: the list of trading days, then one day's trades
Commentary storage	<code>lib/ai-commentary/store.ts</code>	Writes each note with its prompt key + version

Concept	Method (file)	What it does
Decision storage	<code>lib/auto-trade/store.ts</code>	Every AI pass with provider, model, and token counts

The maths — how a pick is graded

For each pick, take every 5-min candle **from the moment it was suggested** (never earlier — no credit for moves it didn't call) to the close:

```
maxUpPct   = (highest high - spot at suggestion) ÷ spot × 100
maxDownPct = (lowest low - spot at suggestion) ÷ spot × 100
closePct   = (day's close - spot at suggestion) ÷ spot × 100
```

Read them direction-aware: for a bullish pick `maxUpPct` is "best case" and `maxDownPct` is "worst pain"; for a bearish pick it's the mirror. A bearish pick with `maxDownPct -11.5%` (PATANJALI, 15 Jul) was a monster; a bearish pick whose `closePct` is *positive* (TATAELXSI, same day) went the wrong way. These three numbers per pick are the raw material of every filter decision this project makes.

The end

You now know the whole machine:

Data flows in every 5 minutes → fixed rules shortlist unusual stocks → an AI decides within a narrow, gated mandate → code-enforced limits bound every rupee of risk → everything is recorded → and every evening the system grades its own homework.

The design law from the very first page held through all nine lessons: **the AI proposes, the code disposes.**